Valorant CN 2026 Kickoff Analysis

Andrew Altamirano, Haiwen Lu

Research Questions

1. **What are the most effective agents currently used in Valorant?** We will compile data across many players in the China region to find which agents players most frequently thrive on, and which might be “trap”, or ineffective picks in the current state of the game.
2. **What are the most popular and effective team compositions?** There are four roles that a player will take on in a Valorant team composition, each being more effective at a certain aspect of the game. Thus, there are many different combinations of each role, lending towards different play-styles. We will be looking into which team compositions are most effective overall and against other team archetypes, and which teams prefer to use what compositions.
3. **Who are the most effective players in the region?** We will be looking at a variety of player statistics to determine a player’s effectiveness in-game. We can use vlr.gg’s Rating$^{2.0}$ statistic, along with things like a primary duelist player’s First Kill vs. First Death ratio, for example, to further infer their performance on stage.

Motivation

Valorant is a part of both my and Haiwen’s daily life. For me, it’s mainly as entertainment, but Haiwen aspires to become an analyst for AG, one of China’s current top Valorant teams. This project would be an exercise in match analytics for him, and a deeper dive into the game for me. Hopefully, by expanding upon our knowledge of the game, we will be able to apply this to our own games, and especially, Haiwen’s plans for the future.

Dataset

We have compiled data across many games listed on vlr.gg for China Kickoff 2026. The CSVs are linked below:

- player_agent.csv
- agent_picks.csv
- Team comp wr.csv

Method

1. We will begin by comparing each things like player’s Rating$^{2.0}$ (which provides a “blanket” score pertaining to their performance in game) or ACS (Average Combat Score) to the agents they played. vlr.gg divides this between each agent a player uses, so we can

then calculate the average rating (and other statistics) for the agent. From there, we can draw conclusions about which agents are effective in the current meta (Most Effective Tactic Available), or in other words, the current state of the game.

2. To find the usage rates of each team composition, we will need to manually gather information about each game played. From there, we can find the archetype of each composition utilized by each team, and by repeating this process across many games, we will be able to assemble a general overview of which team compositions are popular, which are actually effective, and how each composition archetype performs against one another.
3. We will use the ratings or ACS of each player, alongside an algorithm we build, to evaluate players' performance pertaining to their primary role. By using other stats rather than solely relying on Rating$^{2.0}$, we will be able to put together a clearer picture of how a player performs in their respective roles. For example, for an entry player, important stats may be their FK/FD ratio or the percentage of time they get traded by a teammate, which isn't something captured just by Rating$^{2.0}$.

EDA results

We got 3 datasets:

1. player_agent.csv— 119 rows, 8 columns
   Each row represents a single player's performance statistics on a specific agent during the VCT CN 2026 Kickoff tournament. If a player used multiple agents, they appear as multiple rows. The columns are: Agent, Player, Rnd (rounds played), ACS (Average Combat Score), K/D (Kill/Death ratio), KAST%, ADR (Average Damage per Round), KPR (Kills per Round)
2. agent_picks.csv— 232 rows, 4 columns
   Each row represents an agent's pick and win rate on a specific map. The columns are: agent (map name, or "overall" for aggregate statistics), agent(agent name), pick%(how often the agent was picked), and win%(win rate when the agent was used).
3. team_comp_wr.csv— 6 rows, 9 columns
   Each row represents a team composition archetype. The columns represent opposing composition archetypes, and the cell values indicate how many times the row composition defeated the column composition. The six archetypes tracked are: Double Duelist, Double Initiator, Double Smokes, DD+DS, DS+DI, and DD+DI.

Our datasets player_agent.csv and team_comp_wr.csv do not contain any missing data. We ensured this was the case by filtering our data in such a way that there was some data collected for each player we included in our dataset, and our team composition winrate dataset encompasses every match played in this event, so there is no missing data. Our agent_picks dataset also was free of missing data because our data source had a winrate statistic for every agent on every map.

Our variables of interest are as follows:

- From player_agent.csv:

- Agent: a nominal categorical variable with observed values of: astra, breach, brimstone, chamber, cypher, fade, gekko, jett, killjoy, neon, omen, phoenix, raze, sage, skye, sova, tejo, veto, viper, waylay, yoru. Values we did not observe were harbor, reyna, iso, clove, miks, deadlock.
- ACS: a numerical variable with 7-number summary as follows:

mean	std. dev	min	q1	median	q3	max
195.39	31.39	124.00	171.05	199.00	215.60	276.70

- KAST%: a numerical variable (with % symbols, dropped in proocessing) with 7-number summary as follows:

mean	std. dev	min	q1	median	q3	max
70.50	6.41	50.00	66.50	71.00	74.50	84.00

- From agent_picks.csv:
  - map: a categorical variable with observed values of: overall, abyss, split, haven, pearl, bind, corrode, breeze.
  - agent: a categorical variable with observed values of: yoru, omen, viper, sova, astra, waylay, neon, fade, killjoy, cypher, brimstone, tejo, raze, breach, skye, jett, veto kayo, chamber, phoenix, gekko, reyna, sage, iso, harbor, vyse, deadlock, clove, miks.
  - pick%: a numerical variable a numerical variable (with % symbols, dropped in processing) with 7-number summary as follows:

mean	std. dev	min	q1	median	q3	max
17.25	27.83	0.00	0.00	0.00	22.0	100.00

    However, this is largely meaningless (without processing) due to the large number of agents who do not get picked and thus have a 0% pickrate.
  - win%: a numerical variable (with % symbols, dropped in processing) with 7-number summary as follows:

mean	std. dev	min	q1	median	q3	max
23.46	29.03	0.00	0.00	0.00	50.00	100.00

    However, this is largely meaningless (without processing) due to the large number of agents who do not get picked and thus have a 0% winrate.
- From team_comp_wr.csv:
  - Winner: a categorical variable with values: Double Duelist, Double Initiator, Double Smokes, DD+DS (Double Duelist and Double Smokes),

DS+DI (Double Smokes and Double Initiator), and DD+DI (Double Duelist and Double Initiator).

- The intersection between the row for the Winner indicated in the first column each column represents the number of times the team archetype indicated in the row won against the team archetype indicated in the column index. Because there are so few values for each column and row, there is little to no meaning in creating a 7-number summary, and could even be misleading.

Data Visualizations:

3 plots for player_agent.csv:

Figure 1: acs_by_agent.png

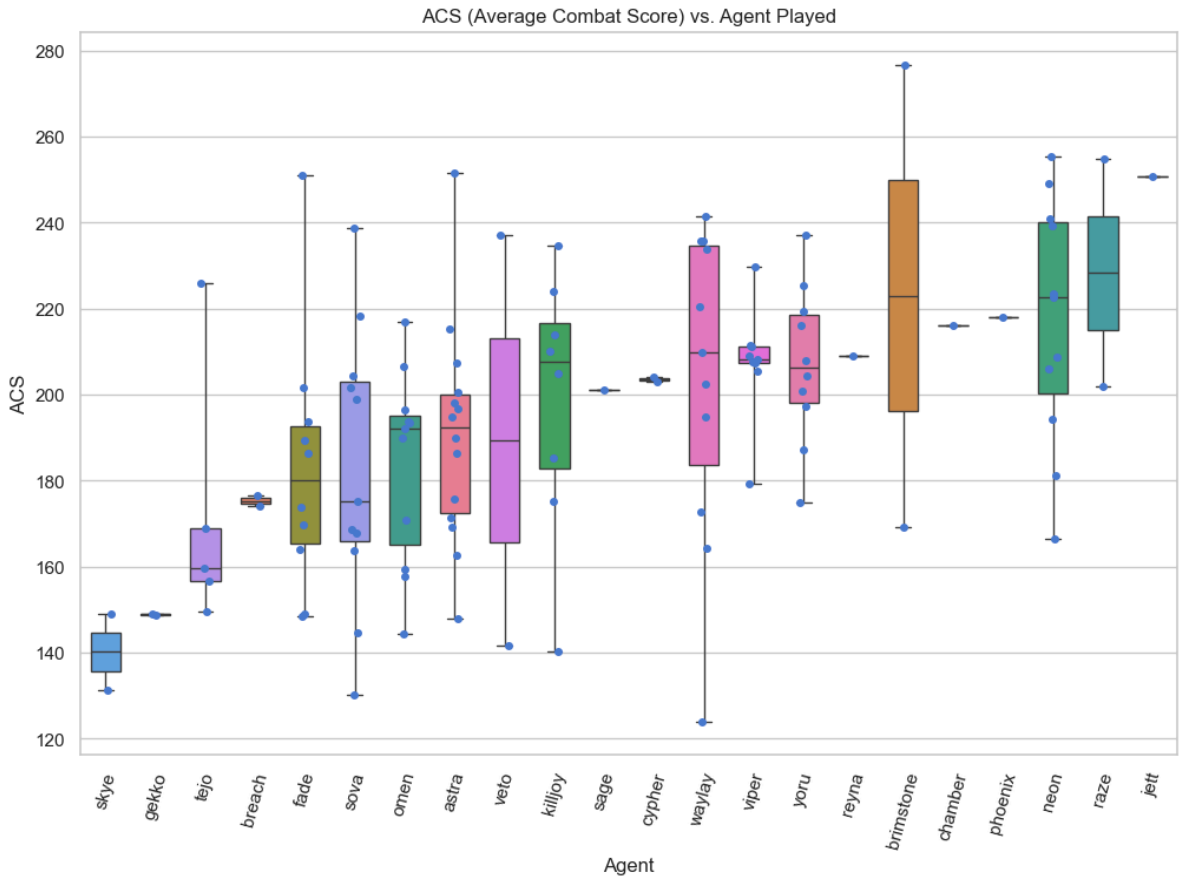

Figure 1: Distribution of player ACS for each agent, ordered by mean ACS, duelists like Jett and Reyna tend to produce the highest combat scores, while smokes and sentinels sit lower.

We chose boxplot. Each box shows the ACS distribution of all players who played that agent, with stripplot dots overlaying individual values. We chose a boxplot because it could show both mean and spread, which a bar chart would hide. The takeaway is that agent role strongly shapes combat output, duelists dominate the top of the chart while support roles cluster lower.

Figure 2: kast_by_agent.png

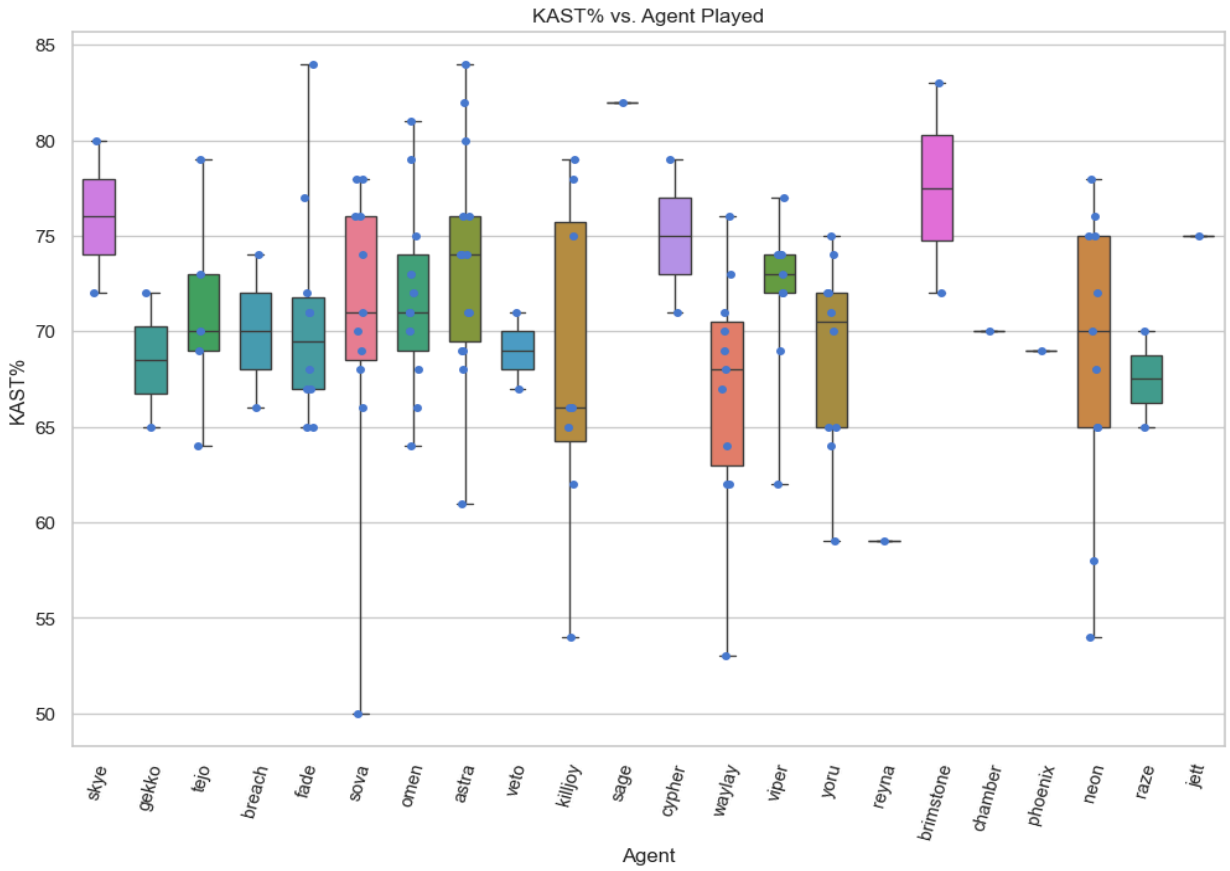

Figure 2: Distribution of KAST% by agent, showing that most agents cluster around 70–80%

KAST% (rounds with a Kill, Assist, Survival, or Trade) measures consistent contribution rather than raw fragging. We used a boxplot here to show the variance shown in the performances better visually than just by showing the data points. Notice that KAST% varies far less across agents than ACS does, indicating that although some agents can't get high acs, they still support the team in another way.

Figure 3: players_acs.png

(The plot is too large to show. Therefore we put it in next page)

Figure 3: ACS across all players, ranked from highest to lowest, highlighting the top performers in the CN Kickoff 2026 tournament.

This horizontal bar chart shows each player's weighted ACS (Average Combat Score), where the weight is round count to account for differences in sample size between players. We chose a bar chart because it cleanly ranks a single metric across a large list of players. The chart directly addresses our third research question: who are the most effective players in the region, by showing that splash and happywei stand out at the top with ACS above 235, while BerLIN sits at the bottom around 130. There is a noticeable drop-off after the top 7 players, suggesting a clear tier separation between the elite performers and the rest of the field.

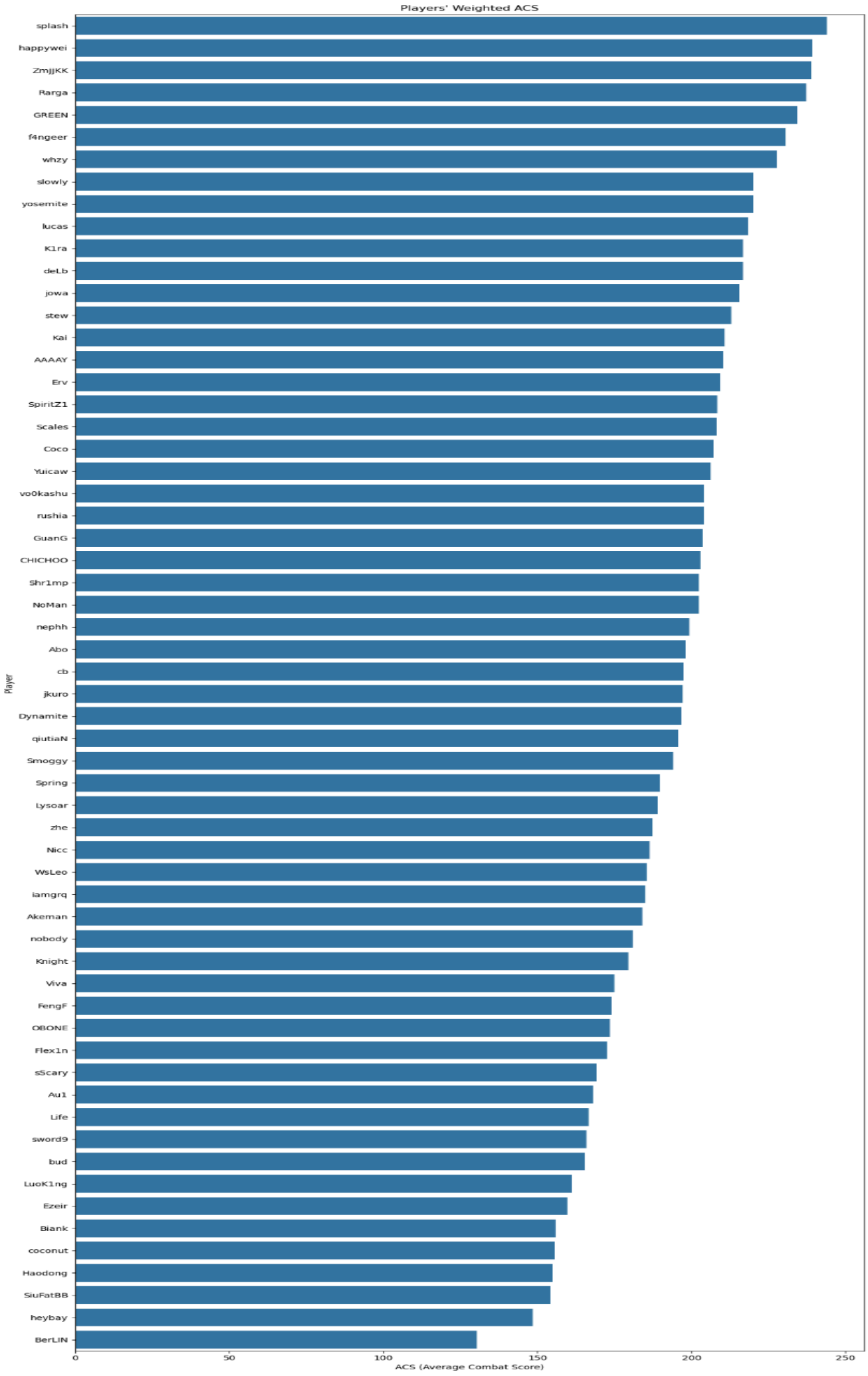
Players' Weighted ACS
splash
happywei
ZmjjKK
Rarga
GREEN
f4ngeer
whzy
slowly
yosemite
lucas
K1ra
deLb
jowa
stew
Kai
AAAAY
Erv
SpiritZ1
Scales
Coco
Yuicaw
vo0kashu
rushia
GuanG
CHICHOO
Shr1mp
NoMan
nephh
Abo
cb
jkuro
Dynamite
qiutiaN
Smoggy
Spring
Lysoar
zhe
Nicc
WsLeo
iamgrq
Akeman
nobody
Knight
Viva
FengF
OBONE
Flex1n
sScary
Au1
Life
sword9
bud
LuoK1ng
Ezeir
Biank
coconut
Haodong
SiuFatBB
heybay
BerLIN
Player
0
50
100
150
200
250
ACS (Average Combat Score)

Figure 4 : agent_winrate_by_map.png

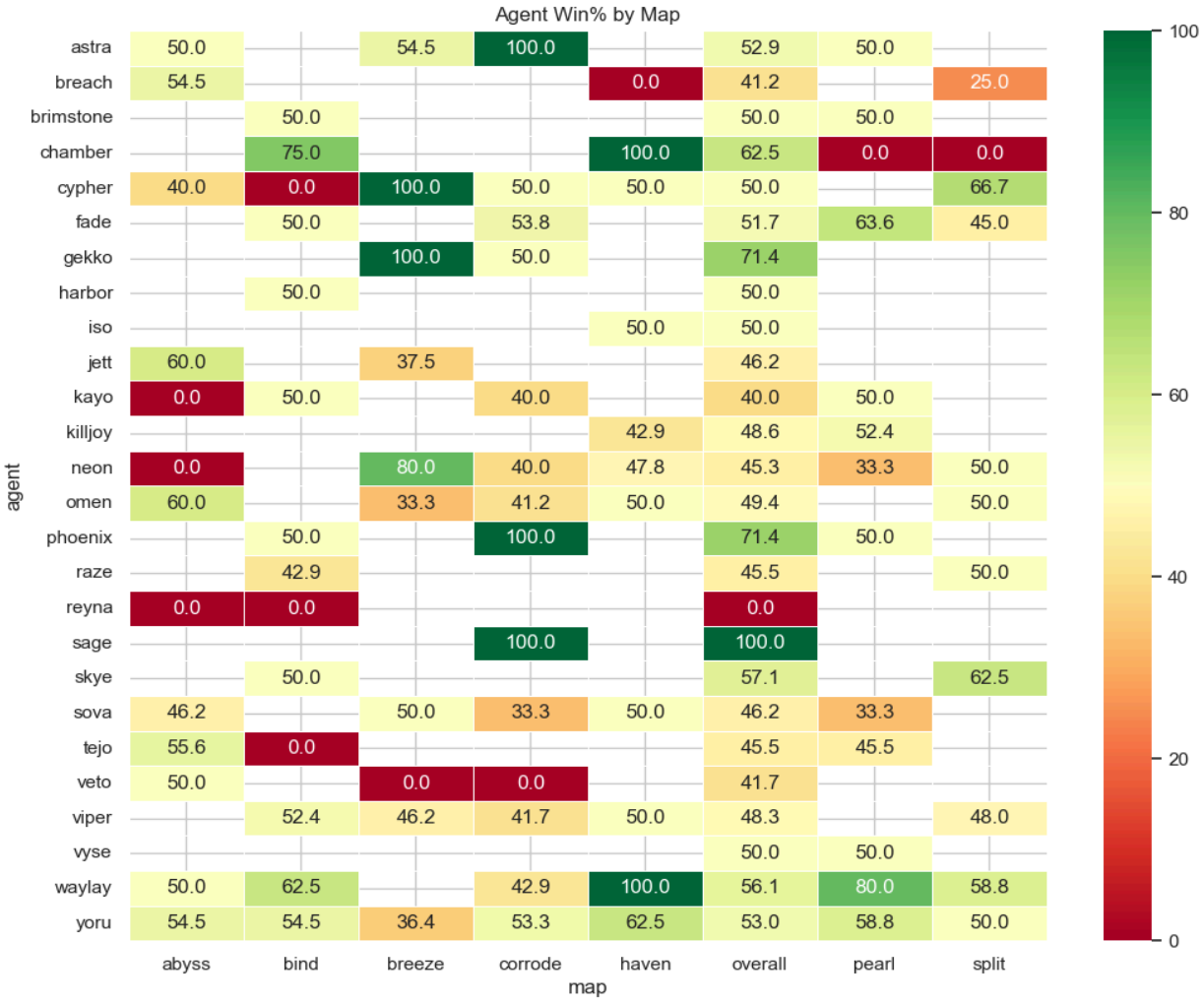

Figure 4: Win rate of each agent on each map; green cells indicate strong performance, yellow cells indicate decent performance, red cells indicate weak ones, and blank cells mean the agent was not picked on that map.

Rows are agents, columns are maps, and color encodes win rate using a red-yellow-green diverging palette. We choose heatmap because the data is intrinsically a 2D matrix and color makes patterns visible more clear than numbers alone. Readers should look for vertical streaks (maps where many agents over- or under-perform) and horizontal streaks (agents with sharply map-dependent strength).

Figure 5: agent_pick_vs_win.png

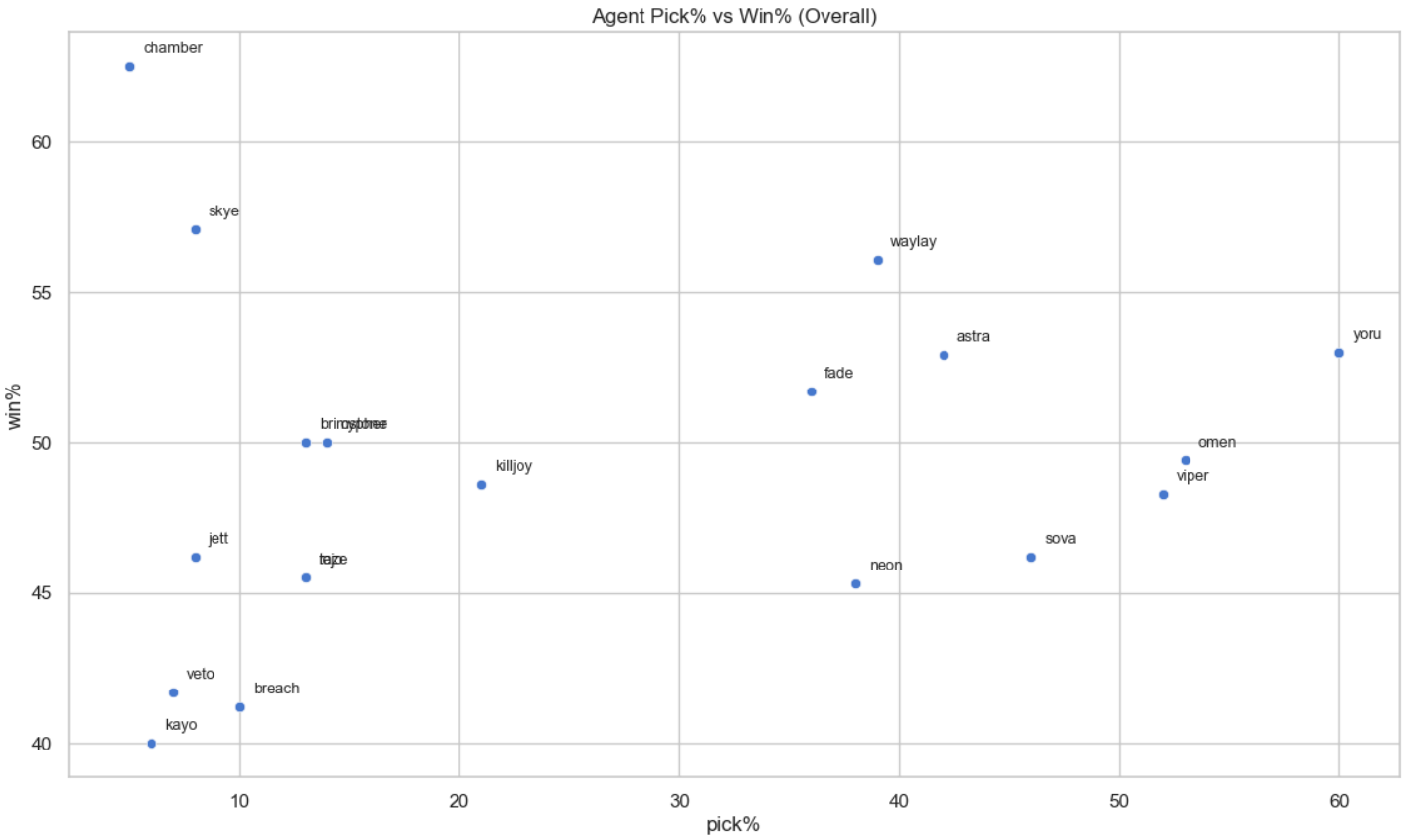

Figure 5: Agent pick rate vs win rate, agents in the upper right are both popular and effective, while agents in the lower right are frequently picked but underperform.

Each point is one agent positioned by their overall pick rate (x) and win rate (y). A scatter plot lets us evaluate two metrics simultaneously and spot agents that are popular and effective versus popular but underperforming. Readers should focus on agents above the 50% win rate line, since those represent the current meta winners, while agents in lower right may be over-picked relative to their actual impact.

2 plots for team_comp_wr.csv:

Figure 6: team_comp_heatmap.png

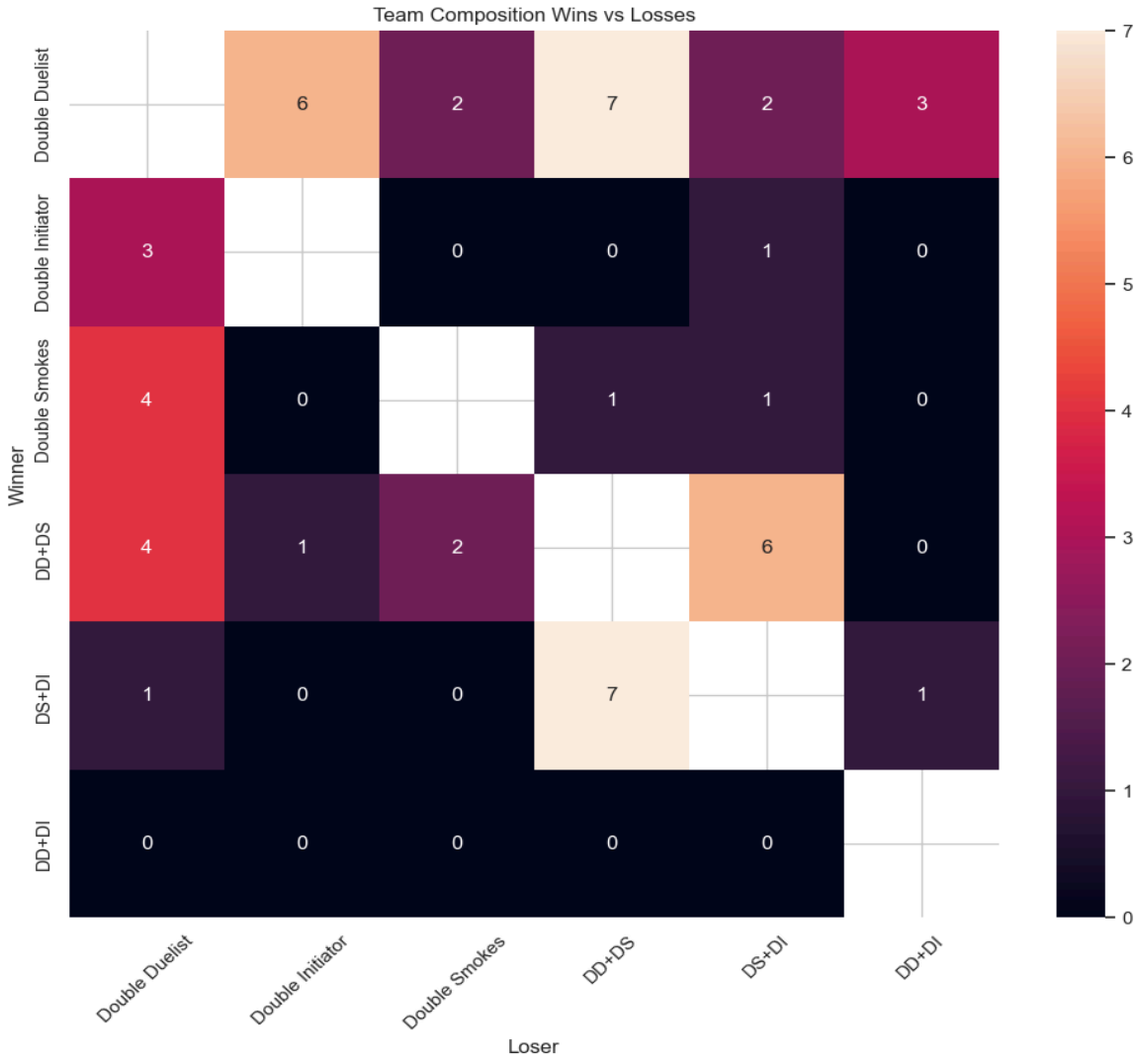

Figure 6: Head-to-head wins between team compositions, rows are winners, columns are losers; lighter cells indicate more wins for that matchup.

This heatmap shows win counts of each composition (rows) against each opponent composition (columns). A heatmap was chosen because it is easy to cross-reference each matchup and find the performance of a team composition at a glance. Notably, Double Duelist beats DD+DS 7 times, and DS+DI also beats DD+DS 7 times, suggesting DD+DS may be a strong composition to play into but struggles in certain matchups. DD+DI sits at 0 wins across the board, indicating it is the weakest archetype in this tournament.

Figure 7: team_wins_vs_losses.png

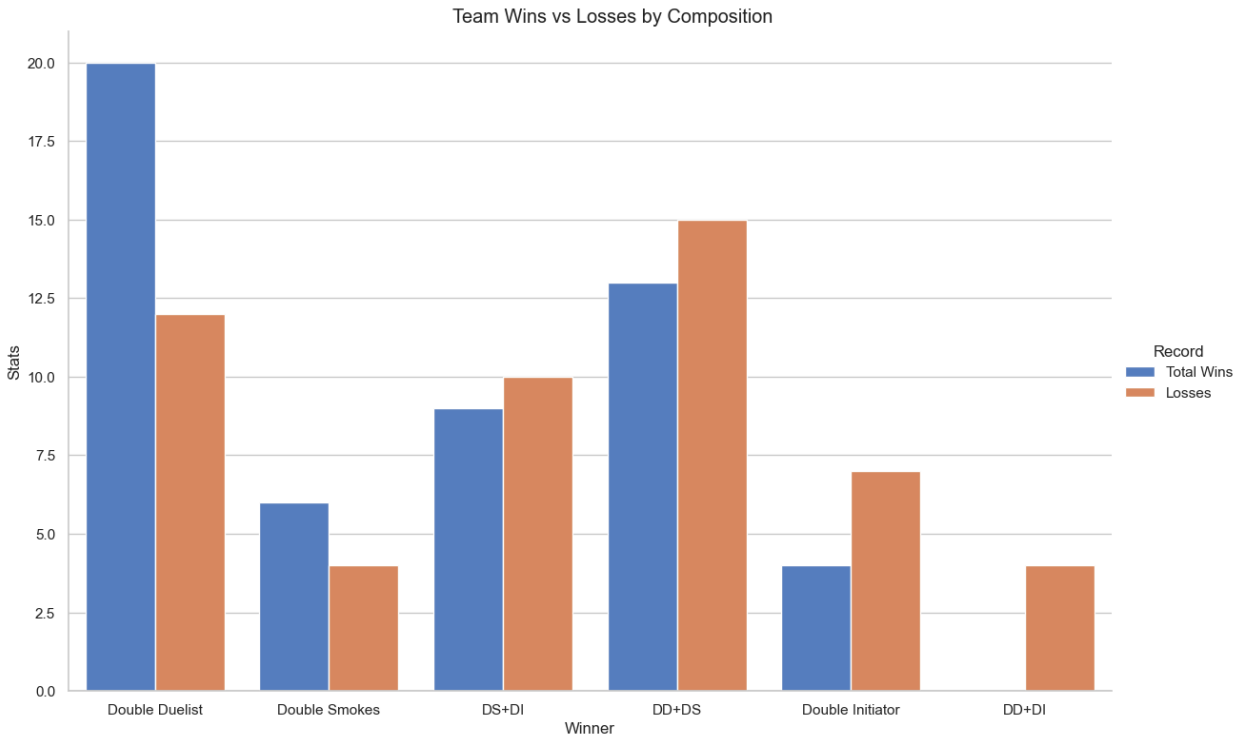

Figure 7: Total wins and losses per team composition, showing Double Duelist as the most-played composition with the highest win total, while DD+DI failed to win a single game

Grouped bars show each composition's total wins and losses side by side. A grouped bar chart was chosen so readers can simultaneously see volume and record. Double Duelist stands out with 20 wins and 12 losses, making it both the most popular and most successful composition. DD+DI on the other hand has 0 wins and 4 losses, suggesting it is either rarely picked or heavily countered in this meta.

Challenge Goals

Messy Data: We will be implementing webscraping to remake some of our datasets, but there is a dataset that may not be possible to scrape, and thus we may have to continue using a dataset we assembled by hand.

New Library: Originally, we were unclear on the specification and did not realize that our data collection had to be done in Python. Now that this has been clarified, we will work after this milestone to implement Python-based webscraping through vlrdevapi. Ideally, we will end up with identical datasets as to what we have right now, but if this results in differently formatted data we will adapt our calculations accordingly. We also have some plots that we cannot show in the document because of their size, and are looking into using libraries like plotly.

Work Plan Evaluation

1. **Data collection and manual compilation**
   Now that we have cleared up the project specifications regarding how we have to put together our datasets, we will need a couple more hours to write the python webscraping code and ensure the datasets are usable for our application.
2. **Data cleaning and formatting**
   Should the datasets be well fitted to our application, we may need an hour or two to ensure the datasets are cleaned correctly.
3. **Agent and player effectiveness analysis**
   We still need to implement the rest of our player evaluation code and thus will need to work for some time toward this goal.
4. **Team composition analysis and visualization**
   With our static visualizations done, we just need to make a couple more visualizations once we have worked with plotly, so we will need a ~5 hours for this.

Testing

In order to ensure our calculations in our code worked as expected, we calculated a handful of statistics by hand and compared them against the figures calculated by our code. All the numbers were calculated in the same way, so by verifying a subset of the values, we verified that our code was functioning correctly, and we were getting the correct numbers through our dataframe operations.

Collaboration

We did not consult any people or resources outside of the course staff and our team members for this project